



Objective & Lecture Mapping: In Lecture 03, we learned that a mathematically perfect "Table-Driven Agent" is impossible to build due to combinatorial explosion. Instead, we must write Agent Programs. Today, you will implement a **Simple Reflex Agent** using Condition-Action rules, observe its fatal flaws in a partially observable environment, and then upgrade it to a **Model-Based Agent** that maintains an internal memory state.

Part 1: Practical Implementation — Coding the Architectures

Step 1.1: Setting the Trap (Partial Observability)

To see why Simple Reflex agents fail, we must handicap your agent's sensors. We are moving from a Fully Observable world to a Partially Observable one.

1. Open your `visual_grid_game.py` from Lab 01.
2. Locate the `get_percept(self)` method.
3. Modify it so it **no longer returns** the exact global coordinates (`agent_pos`). Instead, it should only return local booleans, e.g., `{'wall_ahead': True/False, 'food_here': True/False}` based on checking the adjacent cell in its current facing direction.

Step 1.2: The Simple Reflex Agent (Implementation & Failure)

1. Create a new class called `SimpleReflexAgent` .
2. Implement a `sense_and_act(self, percept)` method that uses strictly IF-THEN logic (Condition-Action rules). *Do not use an `__init__` method to store history.*
Example Logic: IF `food_here` THEN `suck`; IF `wall_ahead` THEN `turn_left`;
ELSE `move_forward`

3. Run the simulation. **Observe:** Watch the agent get trapped in a corner or a U-shaped wall, infinitely repeating a cycle (e.g., turn left, step forward, hit wall, turn left...).

Step 1.3: The Model-Based Agent (Memory & State)

1. Create a new class called `ModelBasedAgent`.
2. Implement an `__init__(self)` method to initialize an internal memory state (e.g., `self.visited_cells = set()` or a relative movement tracker).
3. In `sense_and_act(self, percept)`, first **update the state** (Transition & Sensor Model) by recording the current percept and the last action taken.
4. Update your IF-THEN rules to query the memory.
Example Logic: IF `wall_ahead` AND `left_is_visited` THEN `turn_right`
5. Run the simulation. **Observe:** The agent should now remember where it has been, realize it is in a loop, and choose an alternate path to escape.

Part 2: Theoretical Evaluation & Lecture Mapping

Based on your practical observations above, answer the following questions to map your code directly to the architectural theories covered in Lecture 02.

1. (Remember) According to Lecture 02, why is it impossible to program a mathematically perfect "Table-Driven Agent" for complex environments like Chess? What happens as the agent's lifetime increases?

A table-driven agent needs one entry for every possible percept sequence it could ever encounter, mapped to an action. As the agent's lifetime increases, the number of possible percept histories grows exponentially - this is combinatorial explosion. For something as complex as chess, the number of possible board states/percept sequences vastly exceeds the number of atoms in the observable universe, so the table could never be built or stored, let alone hand coded.

2. (Understand) Look at the code you wrote for your `SimpleReflexAgent`. Identify and explain the specific lines of code that represent the "Condition-Action Rules" discussed in the lecture.

Each if/elif branch in `sense_and_act` is one condition-action rule: the condition is the boolean check on the current percept (`percept['food_here']`, `percept['wall_ahead']`), and the action is the return value ('suck', 'turn_left', 'move_forward'). There's no `_init_` storing history - each call is a fresh, memoryless mapping from percept to action, exactly matching the "IF condition THEN action" definition from the lecture.

3. (Analyze) Your `SimpleReflexAgent` likely got stuck in an infinite loop during Step 1.2. Based on the lecture, analyze exactly why this happened. How did the combination of "Partial Observability" and a lack of "Percept History" cause this failure?

Partial observability means the percept at any instant only shows what's immediately adjacent (`wall_ahead`, `food_here`) - it hides the agent's actual position and where it's already been. Because the agent has no percept history, two genuinely different world states (e.g., "corner A" and "corner B") can produce an identical percept ("wall ahead"). Since the rule set maps percept → action deterministically, the agent applies the exact same action every time it sees that percept - `turn_left`, hit wall, `turn_left` - regenerating the same percept forever. Without memory to distinguish "I've been here before" from "this is new," it cannot

4. (Evaluate) In Step 1.3, you added an internal state to your `ModelBasedAgent`. Evaluate how your specific code handles the "Transition Model" (how the world evolves) and the "Sensor Model" (how the agent's actions affect the world).

The sensor model is captured in `update_state`, which interprets the raw percept in light of the last action taken - e.g., it only advances `self.position` if the last action was `move_forward` and no wall was reported, correctly inferring the effect that action had on the world. The transition model is the logic that updates `self.position/self.facing` to predict the new world state before any new rule fires - this happens at the top of `sense_and_act`, so decisions are always made against an up-to-date internal model rather than the instantaneous percept alone. Because `visited_cells` persists across calls, the agent can now detect when `next_cell` has

Finalize and Lock Lab 02 Documentation

